

AI Lab 2 (11/08/26)

Q1. Smart Cleaning Robot: Simple Reflex Agent vs Model-Based Reflex Agent (Reflex Agent with State)

Problem Statement

Design and implement a Python program for a smart cleaning robot operating in a two-dimensional grid environment. The environment contains clean cells, dirty cells, blocked areas, and the robot's starting position. Implement and compare a Simple Reflex Agent and a Model-Based Reflex Agent (Reflex Agent with State).

Problem Representation

The cleaning area is represented by a grid. The robot can move one cell at a time in four directions: up, down, left, or right. It may move only through valid cells and cannot cross blocked areas. When the robot reaches a dirty cell, it can perform a CLEAN action.

Symbol	Meaning
S	Starting position of the robot
D	Dirty cell
C	Clean cell
X	Blocked area / wall

Agent 1: Simple Reflex Agent

The Simple Reflex Agent selects its action only on the basis of the current percept and does not maintain any memory of previously visited or cleaned cells.

- If the current cell is dirty, perform CLEAN.
- If the current cell is clean, move to one valid neighbouring cell according to a fixed rule or movement priority.
- The agent must not store information about previously visited cells.
- The agent may therefore revisit already explored cells.

Agent 2: Model-Based Reflex Agent (Reflex Agent with State)

The Model-Based Reflex Agent maintains an internal state of the environment. It remembers relevant information from previous percepts and uses this state while selecting the next action.

- Maintain a record of visited cells.

- Maintain a record of cells that have already been cleaned.
- Prefer an unvisited valid neighbouring cell whenever possible.
- Use stored state to reduce unnecessary repeated movements.

Implementation Requirements

- Accept a cleaning grid of size at least 10×10 as input.
- The grid must contain one starting position S, multiple dirty cells D, clean cells C, and blocked cells X.
- Implement the Simple Reflex Agent and the Model-Based Reflex Agent as separate functions or classes.
- Run both agents on the same initial grid configuration.
- At each step, display or record the robot position and the action performed.
- Stop when all reachable dirty cells have been cleaned.
- For both agents, report: number of dirty cells cleaned, number of movements, total actions, and number of repeated visits.
- Display the final state of the grid after execution.

Example Grid

```

S C D C X C C D C C
C X C C C C X C D C
D C C X D C C C C C
C C X C C C D X C C
C D C C X C C C C D
C C C D C C X C C C
X C C C C D C C X C
C C D X C C C D C C
C X C C D C C C C C
D C C C C X C C D C

```

Expected Output (For above example)

--- Simple Reflex Agent ---

Dirty cells cleaned: 8

Movements: 46

Total actions: 54

Repeated visits: 17

--- Model-Based Reflex Agent ---

Dirty cells cleaned: 8

Movements: 31

Total actions: 39

Repeated visits: 4

Q2. Robot Delivery Agent: Goal-Based Agent vs Utility-Based Agent

Problem Statement

Design and implement a Python program for an autonomous robot delivery system. The delivery area is represented as a two-dimensional grid containing a robot starting position, a package pickup point, a delivery destination, roads, and blocked areas. Implement and compare a Goal-Based Agent and a Utility-Based Agent.

Problem Representation

The robot starts at S and can move one cell at a time in four directions: up, down, left, or right. The robot may move only through road cells and valid task locations. Blocked cells cannot be crossed.

Symbol	Meaning
S	Robot starting position
P	Package pickup location
G	Delivery destination / goal
R	Road through which the robot can move
X	Blocked area / obstacle

Part A: Goal-Based Agent

The Goal-Based Agent selects actions by considering the desired goal state. In this assignment, the robot has two sequential goals:

- Goal 1: Reach the package pickup location P.
- Goal 2: After picking up the package, reach the delivery destination G.

Use Breadth-First Search (BFS) to generate a shortest route to the current goal. Once the package is picked up, update the current goal from P to G and perform BFS again.

Goal-Based Agent Procedure

- Locate S, P, and G in the grid.
- Set the current goal to P.
- Use BFS to find a shortest reachable route from S to P.
- Move the robot along the generated route and mark the package as picked up.
- Change the current goal to G.
- Use BFS again to find a shortest reachable route from P to G.
- Display both routes, the number of movements for each stage, and the total number of movements.

Part B: Utility-Based Agent

Extend the same delivery environment so that road cells may have different travel characteristics. The Utility-Based Agent should not select a route only because it is the shortest. Instead, it should evaluate alternative routes using a utility/cost function and choose the route that gives the best overall outcome.

Each traversable cell may be assigned one or more of the following values:

- Distance cost (normally 1 per movement)
- Energy consumption
- Risk level
- Traffic or congestion cost

Example cost function: $Cost(route) = w1 \times Distance + w2 \times Energy + w3 \times Risk + w4 \times Traffic$

Consider: $w1=0.4, w2=0.3, w3=0.2, w4=0.1$

BFS should be used to find feasible shortest routes, while the utility/cost function should be used to compare the available alternatives and select the most suitable route.

Utility-Based Agent Procedure

- Use the same start, pickup, and delivery positions as in Part A.
- Assign travel cost information to the traversable cells.
- Find a minimum-cost route from S to P according to the utility/cost function.
- After package pickup, find a minimum-cost route from P to G.
- Display the selected routes, total movements, and total utility/cost.
- Compare the solution with the Goal-Based Agent. The Utility-Based Agent may choose a longer route if its overall cost is better.

Example Grid

```
S R R X R R R R R R
X X R R R X R X R R
R R R X R R R P R R
R X R R R X R R R X
R R R X R R X R R R
X R R R R R R R X R
R R X R X R R R R R
R R R R R X R X R R
R X R R R R R R R R
R R R X R R R R X G
```

Expected Output (For the above grid example)

--- Goal-Based Agent ---

Route to Package: (0,0) -> ... -> (2,7)

Movements to Package: 11

Route to Destination: (2,7) -> ... -> (9,9)

Movements to Destination: 9

Total Movements: 20

--- Utility-Based Agent ---

Route to Package: (0,0) -> ... -> (2,7)

Route to Destination: (2,7) -> ... -> (9,9)

Total Movements: 23

Total Cost: 31.5

The selected route has lower overall utility cost despite having more movements.

Implementation Requirements

- Accept a delivery grid of size at least 10×10 as input.
- The grid must contain one start S, one package location P, one destination G, road cells R, and blocked cells X.
- Implement the Goal-Based Agent and Utility-Based Agent as separate functions or classes.
- The Goal-Based Agent must use BFS for route planning.
- For both agents, display the route from S to P and from P to G.
- Display the number of movements and total route cost, wherever applicable.